

Objetos em Java: Programação Orientada a Objetos

Como pensar de forma fácil na criação de objetos em POO?

A ideia central da Programação Orientada a Objetos

Na Programação Orientada a Objetos (POO), tentamos representar no código elementos que existem no problema que queremos resolver.

Por que POO?

Sem POO, 30 alunos exigiriam 30 variáveis `nome1...nome30`, mais 30 para matrícula, mais 30 para notas. Com POO, você tem um modelo `Aluno` e cria 30 objetos a partir dele. Muito mais organizado!

Por que não usar só variáveis?

Antes de aprender a criar objetos, é importante entender **o problema** que eles resolvem.

Imagine que você precisa guardar os dados de **3 alunos**. Sem POO, você criaria uma variável separada para **cada informação de cada aluno**:

```
1 // Dados isolados: para 300 alunos, seriam 900 variáveis!
2 String nome1 = "Maria"; int mat1 = 101; double nota1 = 8.5;
3 String nome2 = "Joao"; int mat2 = 102; double nota2 = 7.0;
4 String nome3 = "Ana"; int mat3 = 103; double nota3 = 9.0;
```

Listing 1: Sem POO: variáveis soltas para múltiplos alunos

Você pode pensar: *"E se eu usar vetores para agrupar isso?"*

```
1 // Dados do MESMO aluno espalhados em estruturas diferentes
2 String[] nomes = {"Maria", "Joao", "Ana"};
3 int[] matriculas = {101, 102, 103};
4 double[] notas = {8.5, 7.0, 9.0};
```

Listing 2: Sem POO: vetores paralelos (também problemático)

Usar diversos vetores (um para cada atributo) também **não é uma boa opção**. Embora o número de variáveis diminua, os dados de uma mesma entidade perdem a coesão. Se você precisar, por exemplo, ordenar a turma pela nota, terá o trabalho dobrado

de sincronizar as posições nos três vetores ao mesmo tempo. Um pequeno erro de índice e você acaba misturando o nome da Maria com a nota do João!

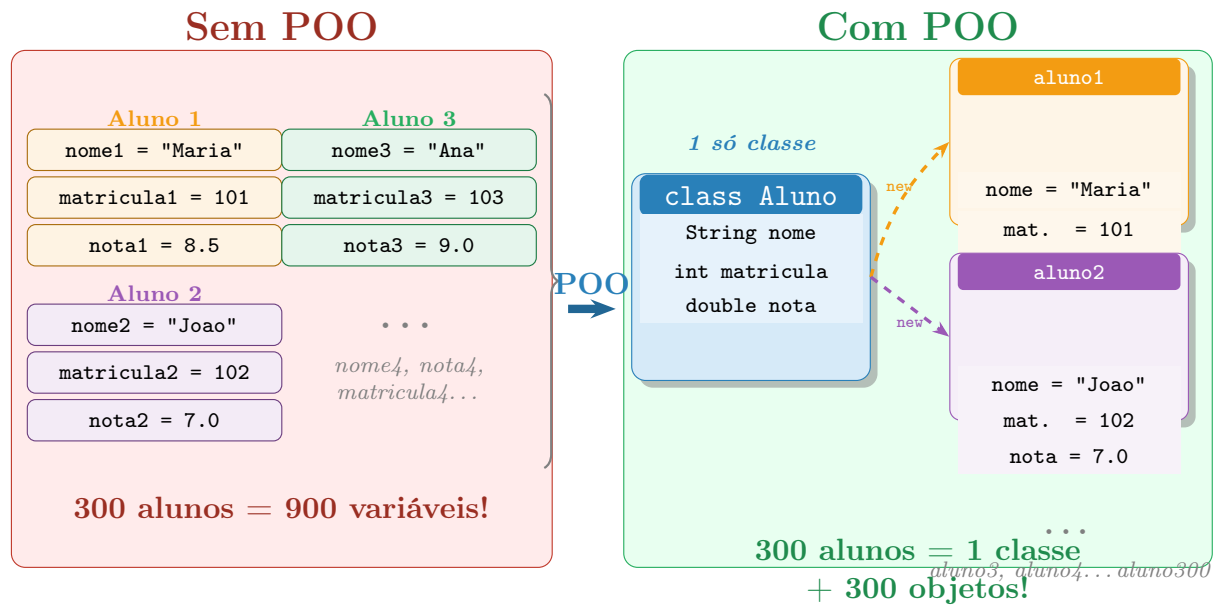


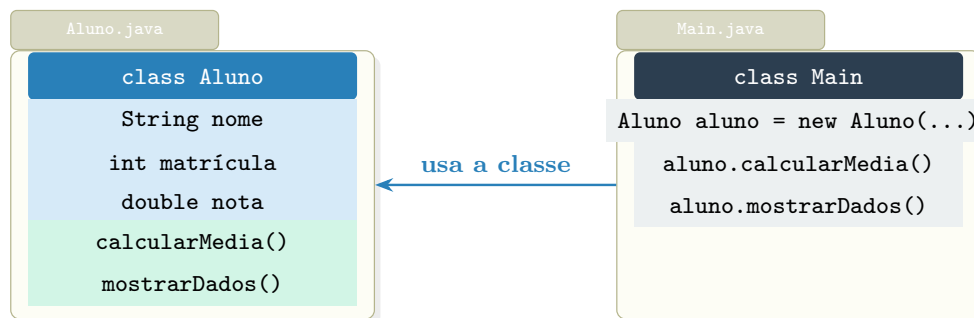
Figure 1: Sem POO: uma variável para cada dado de cada aluno. Com POO: uma classe serve de molde para quantos objetos forem necessários.

Analogia: o formulário

Pense em um **formulário de cadastro** em papel. O formulário **em branco** é a **classe** — o modelo com os campos definidos (nome, matrícula, nota). Cada formulário **preenchido** com os dados de uma pessoa é um **objeto**. Você imprime o mesmo formulário quantas vezes precisar, sem precisar inventar um layout diferente para cada pessoa. É exatamente isso que a POO faz no código.

Resumo

- **Sem POO (Variáveis soltas ou Vetores Paralelos):** Dados ficam espalhados e desconectados → código desorganizado, complexo de manter e propenso a erros.
- **Com POO:** um modelo (classe) + quantos objetos precisar → organizado, coeso, legível e fácil de escalar.



Exemplo coerente: o arquivo `Main.java` cria e usa objetos da classe `Aluno`.

Figure 2: Separação em arquivos: `Aluno.java` guarda o modelo e `Main.java` cria e usa objetos desse modelo.

Dica

Ao ler um problema, destaque os **substantivos principais**. Cada substantivo importante costuma virar uma classe e um arquivo `.java`.

O que é uma classe?

Definição: Classe

Uma **classe** é um modelo, uma forma ou um molde usado para criar objetos. Ela descreve quais informações um objeto terá e quais ações ele poderá realizar.

Analogia: receita de bolo

Uma classe é como uma **receita de bolo**. A receita descreve os ingredientes e o modo de preparo, mas ela própria não é um bolo. Só quando você segue a receita e faz o bolo é que o bolo existe de verdade.

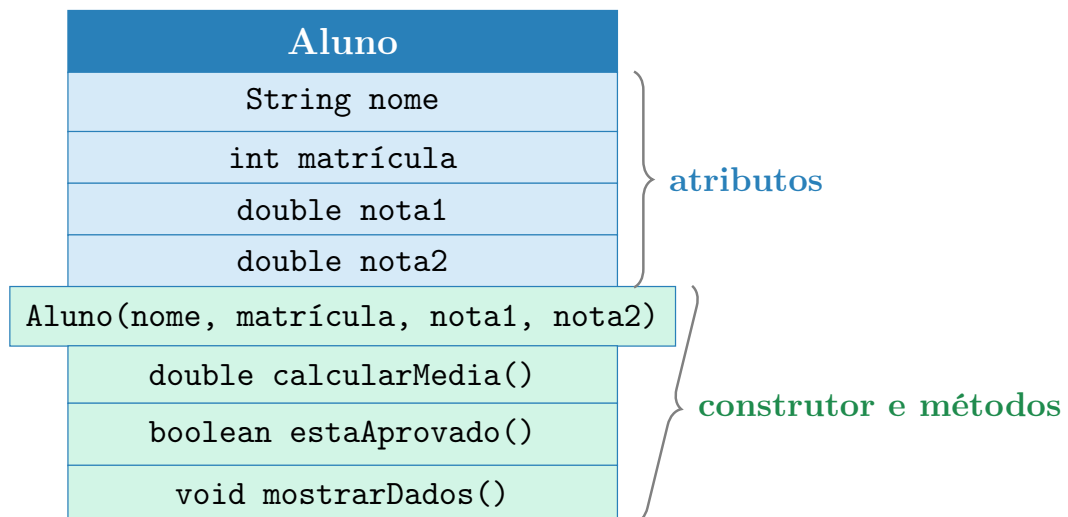


Figure 3: Estrutura de uma classe no estilo UML: atributos (informações) ficam em cima; construtor e métodos (ações) ficam embaixo.

```

1 public class Aluno {
2
3     String nome;           // campo para guardar o nome
4     int matricula;         // campo para guardar a matrícula
5     double nota1;          // campo para guardar a nota 1
6     double nota2;          // campo para guardar a nota 2
7 }

```

Listing 3: Classe Aluno – apenas o modelo

Verifique seu entendimento

A classe Aluno acima representa um aluno real, como “Maria”? Por que sim ou por que não?

O que é um objeto?

Definição: Objeto

Um **objeto** é um elemento concreto criado a partir de uma classe. Com dados reais, que existe na memória do computador.

Analogia: o bolo

Se a classe é a receita, o objeto é o bolo que você fez. Você pode fazer vários bolos com a mesma receita – cada um é independente.

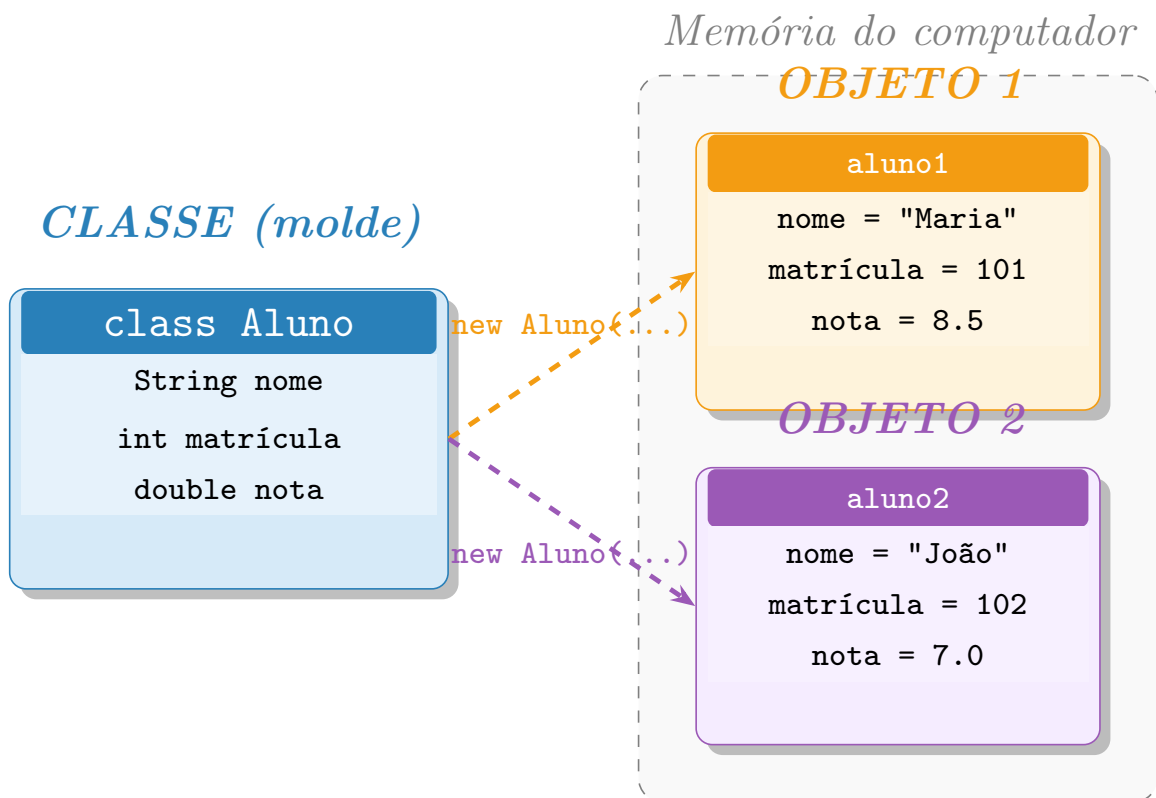


Figure 4: A mesma classe gera múltiplos objetos independentes na memória, cada um com seus próprios dados.

```
1 Aluno aluno1 = new Aluno(); // cria o primeiro aluno na memoria
2 Aluno aluno2 = new Aluno(); // cria o segundo aluno na memoria
```

Listing 4: Criando dois objetos da classe Aluno

Classe é o modelo. Objeto é o elemento criado a partir do modelo.

Verifique seu entendimento

Se você criar três objetos da classe `Caneta`, quantas canetas existirão na memória? Elas compartilham os mesmos dados ou cada uma tem os seus?

Exemplo simples: a classe `Caneta`

Construímos a classe em três etapas:

Etapas 1 – só os atributos:

```
1 public class Caneta {
2
3     String cor;      // a cor da caneta
4     String marca;    // a marca da caneta
5 }
```

Listing 5: Atributos da classe `Caneta`

Etapa 2 – adicionando o construtor:

```
1 public class Caneta {
2
3     String cor;
4     String marca;
5
6     Caneta(String cor, String marca) {
7         this.cor = cor;
8         this.marca = marca;
9     }
10 }
```

Listing 6: Construtor da classe Caneta

Etapa 3 – adicionando os métodos:

```
1 public class Caneta {
2
3     String cor;
4     String marca;
5
6     Caneta(String cor, String marca) {
7         this.cor = cor;
8         this.marca = marca;
9     }
10
11     void escrever() {
12         System.out.println("A caneta está escrevendo.");
13     }
14
15     void mostrarDados() {
16         System.out.println("Cor: " + cor);
17         System.out.println("Marca: " + marca);
18     }
19 }
```

Listing 7: Classe Caneta completa

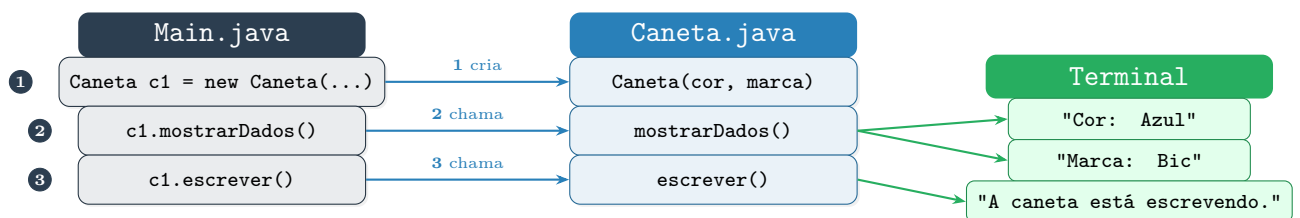


Figure 5: Fluxo de execução: o main cria o objeto e chama seus métodos; cada método gera saída no terminal.

Saída esperada no terminal

```
Cor: Azul
Marca: Bic
```

A caneta está escrevendo.

O que são atributos?

Definição: Atributo

Os **atributos** representam as características ou informações que um objeto possui. São variáveis que pertencem à classe e guardam os dados de cada objeto.

Como identificar atributos

Pergunte: “**Quais informações esse objeto precisa guardar?**” Cada resposta é um atributo.

```
1 public class Aluno {  
2  
3     String nome;           // informação: o nome do aluno  
4     int matricula;         // informação: o número de matrícula  
5     int idade;             // informação: a idade  
6     double nota;           // informação: a nota  
7 }
```

Listing 8: Atributos de um Aluno

Erro comum: confundir atributo com variável local

Atributo pertence ao objeto e existe enquanto o objeto existir. **Variável local** é declarada dentro de um método e some quando o método termina.

Errado:

```
1 void mostrarDados() {  
2     String nome = "Maria"; // variavel local, some logo!  
3 }
```

Correto:

```
1 public class Aluno {  
2     String nome; // atributo, persiste no objeto  
3 }
```

O que é um construtor?

Definição: Construtor

O **construtor** é um método especial executado automaticamente quando usamos o comando **new**. Ele inicializa os atributos do objeto com valores reais.

Analogia: a fábrica

O construtor é como uma linha de montagem. Quando você usa **new**, a fábrica monta o objeto já com os dados que você forneceu.

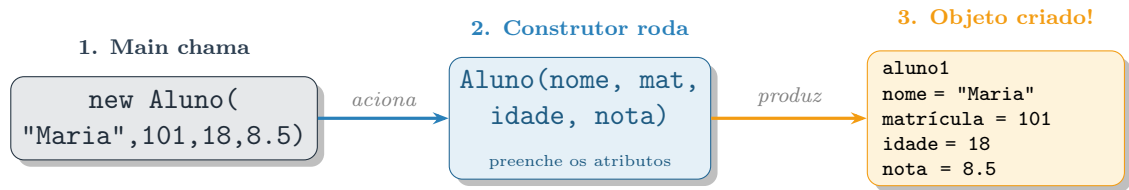


Figure 6: O comando `new` aciona o construtor, que preenche os atributos e entrega o objeto pronto.

```

1 public class Aluno {
2
3     String nome;
4     int matricula;
5     int idade;
6     double nota;
7
8     // O construtor recebe os dados e preenche os atributos
9     Aluno(String nome, int matricula, int idade, double nota) {
10         this.nome = nome;           // guarda o nome no objeto
11         this.matricula = matricula; // guarda a matrícula no objeto
12         this.idade = idade;         // guarda a idade no objeto
13         this.nota = nota;           // guarda a nota no objeto
14     }
15 }
  
```

Listing 9: Construtor da classe `Aluno`

```

1 Aluno aluno1 = new Aluno("Maria", 123, 18, 8.5);
2 //           nome    mat  idade nota
  
```

Erro comum: esquecer de inicializar atributos no construtor

Errado:

```

1 Aluno(String nome) {
2     nome = nome; // isso não faz nada útil!
3 }
  
```

Correto:

```

1 Aluno(String nome) {
2     this.nome = nome; // this.nome = atributo; nome = parametro
3 }
  
```

Para que serve o `this`?

Definição: `this`

A palavra `this` indica o atributo do **próprio objeto atual**, diferenciando-o do parâmetro recebido com o mesmo nome.

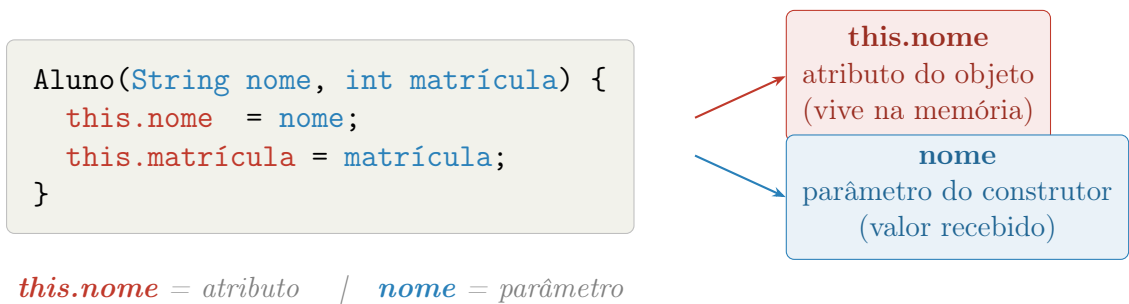


Figure 7: O **this** (vermelho) diferencia o atributo do objeto do parâmetro recebido (azul).

Verifique seu entendimento

O que aconteceria se você escrevesse apenas `nome = nome;` sem o `this`? O atributo do objeto seria preenchido? Teste para descobrir!

O que são métodos?

Definição: Método

Os **métodos** representam as ações que um objeto pode realizar. São funções que pertencem à classe e geralmente usam os atributos do objeto.

Como identificar métodos

Pergunte: “O que esse objeto faz?” Cada resposta (um verbo) é um método: calcular, mostrar, verificar, depositar...

```
1 public class Aluno {
2
3     String nome;
4     double nota1;
5     double nota2;
6
7     Aluno(String nome, double nota1, double nota2) {
8         this.nome = nome;
9         this.nota1 = nota1;
10        this.nota2 = nota2;
11    }
12
13    // Método 1: calcula e retorna a média do aluno
14    double calcularMedia() {
15        return (nota1 + nota2) / 2;
16    }
17
18    // Método 2: verifica se o aluno passou (média >= 7)
19    boolean estaAprovado() {
20        return calcularMedia() >= 7;
21    }
22
23    // Metodo 3: imprime os dados do aluno na tela
```

```

24 void mostrarDados() {
25     System.out.println("Nome: " + nome);
26     System.out.println("Média: " + calcularMedia());
27     if (estaAprovado()) {
28         System.out.println("Situação: Aprovado");
29     } else {
30         System.out.println("Situação: Reprovado");
31     }
32 }
33 }

```

Listing 10: Classe Aluno com métodos

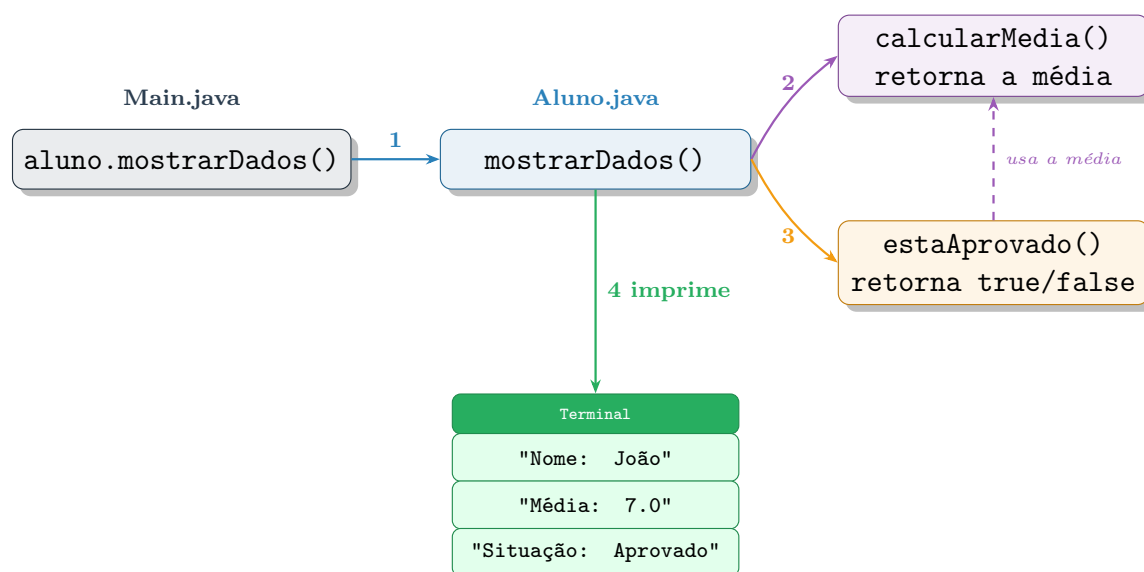


Figure 8: Fluxo de chamadas: `mostrarDados()` chama outros métodos para calcular informações e depois imprimir o resultado.

Saída esperada

```

Nome: João
Média: 7.0
Situação: Aprovado

```

Como diferenciar atributo e método?

	Atributo	Método
O que é?	Uma informação guardada	Uma ação executada
Pergunta	O que este objeto tem ?	O que este objeto faz ?
Exemplo	titular, número, saldo	depositar(), sacar()

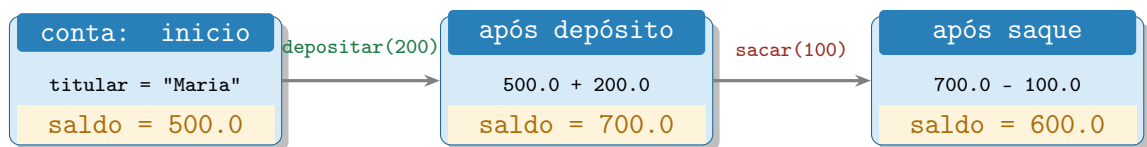
Exemplo: Conta Bancária

```

1 public class ContaBancaria {
2
3     String titular;
4     int numero;
5     double saldo;
6
7     ContaBancaria(String titular, int numero, double saldoInicial)
8     {
9         this.titular = titular;
10        this.numero = numero;
11        this.saldo = saldoInicial;
12    }
13
14    void depositar(double valor) {
15        if (valor > 0) {
16            saldo += valor;
17        }
18    }
19
20    boolean sacar(double valor) {
21        if (valor > 0 && valor <= saldo) {
22            saldo -= valor;
23            return true;
24        }
25        return false;
26    }
27
28    void mostrarDados() {
29        System.out.println("Titular: " + titular);
30        System.out.println("Número: " + numero);
31        System.out.println("Saldo: " + saldo);
32    }
33 }

```

Listing 11: Classe ContaBancaria completa



O objeto continua sendo o mesmo; o valor do atributo `saldo` é atualizado.

Figure 9: Evolução do estado do objeto conta: somente o atributo `saldo` muda a cada operação.

```

1 public class Main {
2     public static void main(String[] args) {
3         ContaBancaria conta = new ContaBancaria("Maria", 1001,
4         500.0);
5         conta.depositar(200.0);
6     }
7 }

```

```

5         conta.sacar(100.0);
6         conta.mostrarDados();
7     }
8 }

```

Listing 12: Usando a ContaBancaria no main

Saída esperada

```

Titular: Maria
Número: 1001
Saldo: 600.0

```

Verifique seu entendimento

Com base no diagrama acima, você consegue explicar como o saldo chegou a 600.0?

O que é sobrecarga?

Definição: Sobrecarga

A **sobrecarga** acontece quando temos dois ou mais métodos (ou construtores) com o **mesmo nome**, mas com **parâmetros diferentes**. O Java escolhe automaticamente qual usar com base nos argumentos fornecidos.

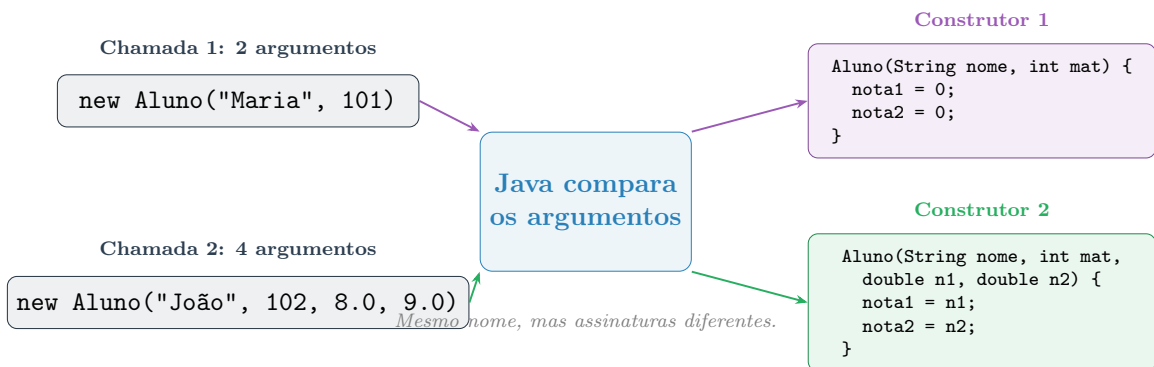


Figure 10: Sobrecarga: o Java analisa a quantidade e os tipos dos argumentos para escolher o construtor correto.

```

1 public class Aluno {
2
3     String nome;
4     int matricula;
5     double nota1;
6     double nota2;
7
8     // Construtor 1: cria aluno sem notas
9     Aluno(String nome, int matricula) {
10         this.nome = nome;
11         this.matricula = matricula;

```

```

12         this.nota1 = 0;
13         this.nota2 = 0;
14     }
15
16     // Construtor 2: cria aluno já com notas
17     Aluno(String nome, int matricula, double nota1, double nota2) {
18         this.nome = nome;
19         this.matricula = matricula;
20         this.nota1 = nota1;
21         this.nota2 = nota2;
22     }
23
24     void mostrarDados() {
25         System.out.println("Nome: " + nome + " | Matrícula: " +
matricula);
26         System.out.println("Nota 1: " + nota1 + " | Nota 2: " +
nota2);
27     }
28 }

```

Listing 13: Sobrecarga de construtores na classe Aluno

Verifique seu entendimento

Se você chamar `new Aluno("Ana", 103, 9.0, 8.5)`, qual construtor o Java vai usar? E se chamar `new Aluno("Pedro", 104)`?

Exemplo com mais de uma classe: Livro e Biblioteca

Como perceber que uma classe precisa de outra?

Observe frases com a ideia de “**tem**”: uma biblioteca **tem** livros, um pedido **tem** produtos, uma turma **tem** alunos. Quando aparece essa relação, uma classe guardará objetos de outra como atributo.

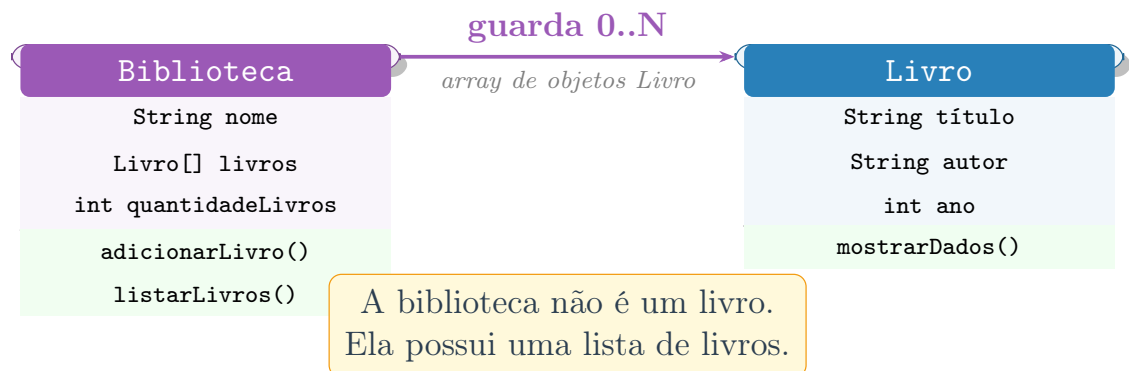


Figure 11: Diagrama de classes: Biblioteca guarda vários objetos Livro em um array, formando uma relação 1:N.

```

1 public class Livro {
2
3     String titulo;
4     String autor;
5     int ano;
6
7     Livro(String titulo, String autor, int ano) {
8         this.titulo = titulo;
9         this.autor = autor;
10        this.ano = ano;
11    }
12
13    void mostrarDados() {
14        System.out.println("Título: " + titulo);
15        System.out.println("Autor:   " + autor);
16        System.out.println("Ano:      " + ano);
17    }
18 }

```

Listing 14: Classe Livro

```

1 public class Biblioteca {
2
3     String nome;
4     Livro[] livros;           // array de objetos da classe Livro
5     int quantidadeLivros;
6
7     Biblioteca(String nome, int capacidade) {
8         this.nome = nome;
9         this.livros = new Livro[capacidade];
10        this.quantidadeLivros = 0;
11    }
12
13    boolean adicionarLivro(Livro livro) {
14        if (quantidadeLivros < livros.length) {
15            livros[quantidadeLivros] = livro;
16            quantidadeLivros++;
17            return true;
18        }
19        return false;
20    }
21
22    void listarLivros() {
23        System.out.println("=== " + nome + " ===");
24        for (int i = 0; i < quantidadeLivros; i++) {
25            livros[i].mostrarDados();
26            System.out.println("---");
27        }
28    }
29 }

```

Listing 15: Classe Biblioteca – contam um array de objetos Livro

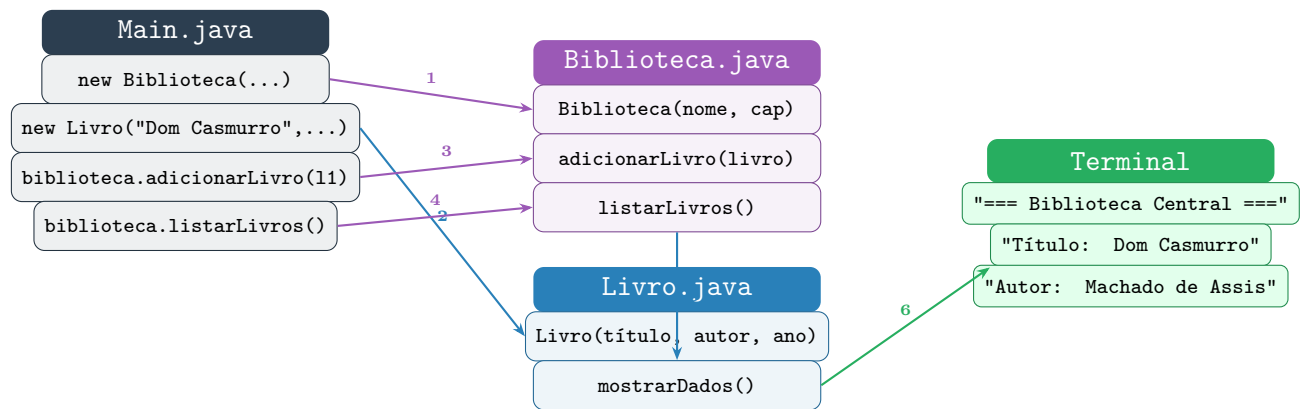


Figure 12: Fluxo completo: Main coordena Biblioteca, que por sua vez chama métodos de Livro.

```

1 public class Main {
2     public static void main(String[] args) {
3         Biblioteca biblioteca = new Biblioteca("Biblioteca Central"
4         , 3);
5         Livro livro1 = new Livro("Dom Casmurro", "Machado de Assis"
6         , 1899);
7         Livro livro2 = new Livro("O Cortiço", "Aluísio Azevedo",
8         1890);
9         biblioteca.adicionarLivro(livro1);
10        biblioteca.adicionarLivro(livro2);
11        biblioteca.listarLivros();
12    }
13 }
  
```

Listing 16: Usando Livro e Biblioteca juntos no main

Erros comuns de iniciantes

Evite esses erros!

1. **Criar tudo dentro do main.** O main deve apenas criar objetos e chamar seus métodos. A lógica fica nas classes.
2. **Criar atributos, mas não criar métodos.** Se os dados não são manipulados por métodos, a classe não tem comportamento.
3. **Criar métodos que não usam os atributos do objeto.** Isso sugere que o método está na classe errada.
4. **Esquecer de inicializar os atributos no construtor.** Atributos não inicializados recebem null, 0 ou false.
5. **Confundir classe com objeto.** Classe é o modelo. Objeto é o elemento criado a partir do modelo.
6. **Esquecer o this.** Sem this, o Java não sabe a diferença entre o atributo e o parâmetro.

Resumo dos conceitos

Conceito	Explicação simples	Pergunta-chave
Classe	Modelo usado para criar objetos.	O que é essa coisa?
Objeto	Elemento concreto criado a partir de uma classe.	Como esse elemento existe?
Atributo	Informação guardada pelo objeto.	O que ele tem ?
Construtor	Inicializa o objeto quando ele é criado.	Como ele nasce ?
Método	Ação que o objeto pode executar.	O que ele faz ?
Sobrecarga	Mesmo nome, parâmetros diferentes.	Quantas formas?
this	Indica o atributo do próprio objeto.	De qual objeto?

Atividade prática

Escolha um dos temas abaixo e modele as classes necessárias. Para cada tema, responda as quatro perguntas fundamentais **antes** de programar.

1. Sistema de cadastro de alunos em uma turma.
2. Sistema de contas bancárias em um banco.

3. Sistema de livros em uma biblioteca.
4. Sistema de jogadores em um time.
5. Sistema de produtos em uma loja.

Para cada tema, responda antes de programar:

- Quais **classes** (e arquivos `.java`) serão criados?
- Quais **atributos** cada classe terá?
- Quais **construtores** serão necessários?
- Quais **métodos** cada classe terá?
- Como os **objetos** serão criados e usados no `main`?